Octaby v2: Product Requirements

Oct 23, 2026 · @H

This PRD defines what Octaby v2 must do and how success is measured. Implementation details (hardware, architecture, timeline) live in Doc.

Updated Oct 23: the system now runs on a laptop (YOLO + MediaPipe) driving an Arduino over USB. Where the build plan still describes the earlier iPhone and LiDAR design, this PRD wins.

Problem and opportunity

Blind and low-vision people can usually learn *that* an object is nearby, but not get their hand onto it without searching by touch, which knocks things over and risks contact with sharp or hot items.

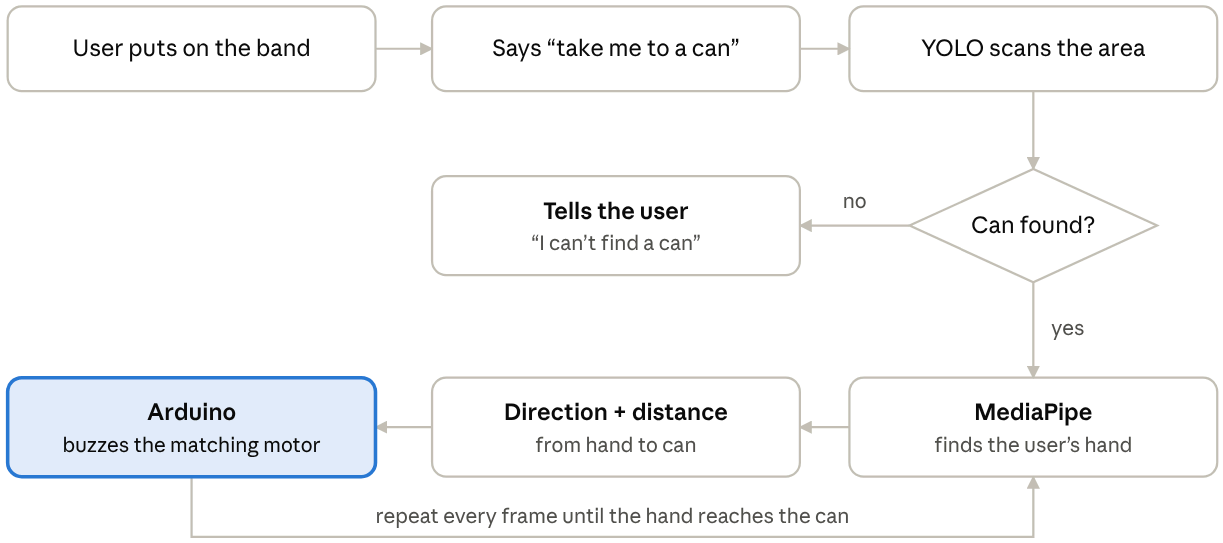
- Existing tools (white canes, smart canes, Seeing AI, Be My Eyes) focus on navigation or one-time scene description.
- None give continuous, hands-free correction during the reach itself, the last 50 cm.
- Audio-only tools fail deafblind users and compete with sounds people need to hear.

Opportunity: a phone that sees plus a wristband that steers can turn "it's on the table" into "it's in my hand" in seconds, privately and offline.

How it works

The user asks for an object by voice; YOLO either finds it and the band steers the hand there, or the app says plainly that it isn't there.

If YOLO finds the can, the band buzzes until the hand gets there



core flow · find, guide, or say not found

1. The user puts on the band and says "take me to a can."
2. YOLO scans the camera view for a can for a few seconds.
3. **Not found:** the app says "I can't find a can" and stops. No guessing, no guiding toward something else.
4. **Found:** MediaPipe finds the user's hand in the same frame, and the laptop works out the direction and distance from hand to can.
5. The laptop sends a short command over USB to the Arduino, which turns on the vibration motor for that direction.
6. Steps 4 and 5 repeat every frame until the hand reaches the can.

YOLO and MediaPipe run on the laptop, not the Arduino, since an Arduino is too small to run either model; the Arduino only receives "buzz this motor" commands. The spec uses vibration motors rather than servos, because servos turn rather than vibrate (v1 tried servos and moved to vibration motors).

Goals, non-goals and success metrics

The product succeeds if a blindfolded first-time user grabs a named object within 15 seconds, without touching anything else, in at least 8 of 10 tries.

Goals

1. Guide a user's hand onto a spoken, named object using haptics alone.
2. Tell the user plainly when the object isn't there, instead of guessing.
3. Be learnable in under 60 seconds.
4. Warn the user before their hand reaches a hazard.

Non-goals (v2)

- Walking navigation or obstacle avoidance; canes and existing apps cover this.
- Reading text, labels or currency.
- Android support.
- Medical-device claims or certification.

Success metrics (targets are team-set starting points; adjust after first tests)

Metric	Target	How measured
Grab success rate	80% or more on first attempt	Blindfolded trials, 5 objects
Time to grab	Median under 15 s from command	Stopwatch or app log
Wrong contacts	Under 1 per 10 trials	Observer tally
Learning time	Under 60 s of training mode	New users, timed
Guidance latency	Under 100 ms from hand move to buzz change	App log timestamps
Hazard warnings	100% of trials with a knife or scissors in path	Scripted trials

Target users and personas

The primary user is a blind person doing everyday tasks at home; two secondary personas widen the story.

Persona	Situation	Need	What Octaby must get right
Maya, 34, blind since birth (primary)	Cooks, works from home, uses VoiceOver daily	Grab the right item fast without sweeping the counter	Speed, accuracy, works with VoiceOver
Daniel, 58, temporary vision loss	Recovering from eye surgery or using dilating drops	Short-term help with no learning curve	Training under 60 s, simple voice commands
Lena, 41, deafblind	Cannot use audio-only tools	Directions through touch alone	Every cue available as haptics, no audio required

Personas are illustrative, built from the use cases in the build plan, not from user research. Replace them with real interviewees when you test.

Functional requirements

P0 items define the hackathon MVP; P1 ships if the core loop works by hour 10; P2 is post-hackathon.

ID	User story	Priority	Owner	Acceptance criteria
FR-1	As a user, I say "take me to a can" and the system starts looking	P0	Harim (UI app)	Command recognized; app confirms out loud within 1 s
FR-2	As a user, the system scans the area for the object I asked for	P0	Harim, Rizwan (YOLO)	YOLO detects "can" at 0.5 confidence or more for 3 frames in a row
FR-3	As a user, if the object isn't there, I'm told straight up	P0	Harim, Rizwan (YOLO) + Harim (UI)	After about 5 s with no match, app says "I can't find a can" and stops
FR-4	As a user, the system knows where my hand is	P0	Unassigned (MediaPipe)	MediaPipe returns the hand's position in the same frame as the can

ID	User story	Priority	Owner	Acceptance criteria
FR-5	As a user, my hand is steered toward the object	P0	Unassigned (guidance)	Direction (up, down, left, right) and distance computed every frame from hand and can positions
FR-6	As a user, I feel which way to move	P0	Azfar (hardware)	Arduino receives a command over USB serial and buzzes the right motor within 50 ms
FR-7	As a user, I'm told when I've reached the object	P0	Azfar + guidance owner	All motors give the "grab" pattern when hand and can overlap
FR-8	As a user, I'm told when the camera loses my hand	P1	Guidance owner + Harim (UI)	Buzzing stops within 1 s; app says "I can't see your hand"
FR-9	As a user, I can ask for other objects (bottle, cup, phone)	P1	Harim, Rizwan (YOLO)	Same flow works for at least 5 object types
FR-10	As a user, I'm warned before I touch something dangerous	P2	YOLO + UI	Knife or scissors near hand path triggers a spoken warning
FR-11	As a user, the band works without a cable	P2	Azfar	Bluetooth link replaces USB serial

Heads-up for FR-2: standard YOLO models are trained on 80 COCO classes, which include bottle and cup but not can. The YOLO owners need either YOLO-World with "can" or "soda can" as a custom class, or a quick fine-tune on photos of cans. Decide this in the first hour.

Non-functional requirements

Area	Requirement
Latency	Hand movement to motor change under 150 ms end to end; YOLO and MediaPipe each at 10 fps or more
Offline	Whole flow works with no internet
Privacy	Camera frames stay on the laptop and are never saved by default
Accessibility	Every step works by voice and haptics; nothing requires seeing a screen
Comfort	Band fits most wrists; no exposed wires against skin
Safety	System never says an area is "safe"; it reports only what it detected
Hardware	Laptop with webcam, Arduino (Uno, Nano or similar) over USB, 4 vibration motors with driver transistors
Portability	Code runs on Windows and macOS laptops; serial port and camera index set in a local config file, not hard-coded

Release scope

The hackathon build ships FR-1 to FR-7; everything else is gated on that loop working end to end.

Release	Includes	Exit criteria
MVP (hackathon)	FR-1 to FR-7	Blindfolded judge says "take me to a can" and grabs it on the first try; with no can present, the system says so

Release	Includes	Exit criteria
1.1 (same event if time allows)	FR-8 and FR-9	Same flow works for 5 object types; hand-lost cue fires every time
2.0 (after the event)	FR-10, FR-11, real-user testing with a blindness organization such as CNIB	Success metrics met with blind and low-vision testers, not just blindfolded teammates

Team and ownership

Each person works in their own folder and Git branch; the pieces meet through fixed interfaces defined in the repo's AGENTS.md files.

Area	Owner	Repo folder	Branch
Hardware: Arduino, motors, band, serial protocol	Azfar	`hardware/`	`feat/hardware`
YOLO object detection	Harim, Rizwan	`vision/yolo/`	`feat/yolo-harim`, `feat/yolo-rizwan`
UI app: voice in, speech out, main loop	Harim	`app/`	`feat/app`
MediaPipe hands + guidance math	Unassigned	`vision/hands/`, `guidance/`	`feat/hands`, `feat/guidance`
Shared interfaces	Whole team, by agreement	`shared/`	`feat/shared` (merged first)

Assumptions, dependencies, open questions

Assumptions

- One webcam sees both the table and the user's hand (laptop camera or a webcam on a stand facing the table).
- Objects sit on a table or counter in reasonable light.

- The user wears the band on one wrist and the system knows which one.
- The Arduino stays plugged into the laptop by USB for the hackathon.

Dependencies

- Ultralytics YOLO (plus YOLO-World or a fine-tune for "can").
- Google MediaPipe Hands for hand landmarks.
- Python 3.11, OpenCV, pyserial, a speech-to-text and text-to-speech library.
- Arduino, 4 vibration motors, transistors and diodes, fabric band.

Open questions

- [ ] Which 5 objects are the MVP demo set?
- [ ] Who owns MediaPipe hand tracking and the guidance math (FR-4, FR-5)? Nobody is assigned yet.
- [ ] Should forward distance be encoded by tempo (current plan) or intensity?
- [ ] Which blindness organization will the team contact for 2.0 testing, and who owns that outreach?